

Model 4 — Adding Listing-Aware Features

M5 Retail Demand Forecasting — Problem Statement 11

Testing the proposed novelty rather than assuming it

NPN AIA Hackathon — St. Joseph's College of Engineering

Generated 2026-08-14 from executed run model_04_tweedie_recency_listing.

Terms used in this report. **RMSE** (Root Mean Squared Error) measures how far predictions land from actual sales, counting big misses much more heavily than small ones — lower is better. **MAE** (Mean Absolute Error) is the plain average size of the miss. **WAPE** expresses total error as a share of total actual demand, which exposes a model that scores well simply by predicting near-zero everywhere. **Leakage** means letting information into the model that would not have existed at the moment the forecast was really made. **SNAP** is the US Supplemental Nutrition Assistance Program, a food-assistance benefit; the dataset records, per state per day, whether it was usable. **Intermittent demand** describes a product that sells on some days and records zero on many others.

Objective

The project's proposed novelty was 'Listing-Aware + Recency-Aware Demand Forecasting'. This experiment tests the listing half of that claim against measurement.

What we did

Model 3 plus feature group D: days_since_first_listing and pre_listing. The idea, taken from the EDA, is that many early zeros are not weak demand at all — the product simply was not on the shelf yet — and that a model told which zeros are which should stop treating them as evidence of low demand.

The foundation stage had already measured something important about this feature, and it was known before this experiment ran: rows flagged pre_listing have a **100.00% zero-sales rate**, confirming the structural claim — but at this forecast origin **0% of rows are pre-listing**, because by 2016 every product in the catalogue has long since been listed. The feature therefore has nothing to act on at prediction time; it can only shape what the model learns from older training rows.

Data used

Training rows	12,805,800
Training origins	15 (each contributes 30,490 series x 28 days)
Features	32
Feature groups	A, B, C, D, E, F, G
Feature set	base_recency_listing — BASE + Recency + Listing (all 32 features)

Training origins are spaced 28 days apart, so their 28-day target blocks tile the history contiguously and no (series, day) target is counted twice.

Model configuration

Setting	Value
Model	LightGBM
Objective	tweedie (variance_power=1.1)
learning_rate	0.05

Setting	Value
num_leaves	128
max_depth	-1
min_data_in_leaf	100
feature_fraction	0.8
bagging_fraction	0.8
lambda_l2	1.0
seed	42
Boosting rounds	400
Categorical features	12 handled natively by LightGBM
Random seed	42
Training time	116.0s

No early stopping was used, deliberately. Stopping when the validation score stops improving would let the validation window influence a training decision, and the resulting score would flatter itself. A fixed number of rounds keeps the held-out estimate honest.

Validation design

Every model in this project is scored on exactly the same window, with the same metric code, so the comparisons between them are fair by construction.

Forecast origin	d_1913
Days predicted	d_1914 .. d_1941
Dates predicted	2016-04-25 .. 2016-05-22
Horizon	28 days
Series	30,490
Predictions scored	853,720

The model sees no sales at all from the validation window. It is given only the calendar, event, SNAP and price information for those days, all of which are genuinely published in advance.

Results (measured)

Metric	Value
RMSE	2.1210
MAE	1.0319
WAPE	0.7152
Bias (mean predicted – mean actual)	-0.0704
Predictions scored	853,720

Comparison against the previous step

	RMSE	MAE
Model 3 — Tweedie + recency	2.1258	1.0320
Model 2 — Tweedie, base features	2.1256	1.0315

	RMSE	MAE
This model	2.1210	1.0319
Change	-0.0048	-0.0001

Measured verdict: this change **made no meaningful difference**.

What the model learned

The listing insight is real as a *description of the data* and false as a *source of forecasting power for this horizon*. Those are different claims, and the project had been conflating them. A feature that is constant across every row it is asked to predict on cannot separate those rows from one another, no matter how true the underlying observation is.

Leakage checks

The foundation stage established the guarantee this model inherits, and it was verified by experiment rather than asserted: every sales value after the forecast origin was overwritten with an absurd number (9999), the features were rebuilt, and all 32 came back **bit-for-bit identical**. A companion check confirmed the target column did change, proving the corruption really reached the data.

In addition, the training-set builder refuses to run if any training row targets a day inside the validation window — that is an assertion in code, not a convention.

Limitations

- `pre_listing` and `price_is_missing` were measured to be identical at every origin tested, so one is redundant.
- The result is specific to a 2016 forecast origin. On a horizon containing genuine new-product launches the feature could matter more.

Conclusion and next step

Measured result: adding listing features made no meaningful difference. The next experiment changes the model's structure rather than its inputs.